

Iniziato	venerdì, 4 luglio 2025, 14:14
Stato	Completato
Terminato	venerdì, 4 luglio 2025, 15:18
Tempo impiegato	1 ora 4 min.
Valutazione	5,00 su un massimo di 12,00 (41,67%)

Domanda **1**

Risposta errata

Punteggio ottenuto 0,00 su 3,00

Split in Mergesort

Si consideri l'implementazione delle liste semplici di interi:

```
struct listEl {  
    int      info;  
    struct listEl* next;  
};  
  
typedef struct listEl* list;
```

Si dia un'implementazione di costo $O(n)$ della funzione **split()**:

```
// post: modifica distruttivamente l in modo che l contenga circa la metà  
//       degli elementi della lista data, e ritorna il puntatore alla  
//       lista della seconda metà  
list split(list l)
```

in modo tale che l'implementazione dell'algoritmo Mergesort seguente:

```
// post: ordina distruttivamente l in senso non decrescente  
//       e ritorna il puntatore alla lista ordinata  
list Mergesort(list l) {  
    if (l == NULL || l->next == NULL)  
        return l;  
    else {  
        list m = split(l);  
        l = Mergesort(l);  
        m = Mergesort(m);  
        return merge(l, m);  
    }  
}
```

esegua correttamente.

Nota: nella specifica di **split()**, come in quella di **Mergesort()**, "distruttivamente" significa che NON devono essere allocati nuovi elementi, ma devono essere modificati soltanto i puntatori a next di quelli dati.

Per esempio:

Test	Risultati
Test(1);	Lista data: [4, 1, 3, 9, 1] Lista ordinata: [1, 1, 3, 4, 9]

Risposta (Criteri di penalizzazione: 10, 20, ... %)

Ripristina risposta

L'editor Ace non è pronto. Prova a ricaricare la pagina?
Falling back to raw text area.

```
        return l;  
    }  
  
    else{  
  
        list secondHalf = NULL;  
  
        return secondHalf;  
    }  
  
}
```

	Test	Atteso	Ottenuto	
✘	Test(1);	Lista data: [4, 1, 3, 9, 1] Lista ordinata: [1, 1, 3, 4, 9]	***Errore di esecuzione*** Segmentation fault	✘

Il test è stato arrestato per un errore

Visualizza differenze

Risposta errata

Punteggio di questo invio: 0,00/3,00.

Domanda 2

Risposta errata

Punteggio ottenuto 0,00 su 4,00

Inserimento in una tabella hash ad indirizzamento aperto

Si consideri l'implementazione di una tabella hash a chiavi intere non negative con la tecnica dell'indirizzamento aperto:

```
struct hashFrame {  
    int    dim; // dimensione della tabella  
    int* array; // puntatore alla tabella array[0..dim-1]  
};  
  
typedef struct hashFrame* HashTable;
```

Una tabella viene allocata mediante la funzione (costruttore):

```
// pre: 0 < m  
// post: ritorna una tabella hash di dimensione m a chiavi positive,  
//        inizializzata con tutti -1 che rappresenta l'assenza di chiavi  
HashTable newHashTable(int m)
```

Usando la funzione preimplementata `linearProbing()`:

```
// pre: 0 <= k chiave, 0 <= i iterazione, 0 < m == dimensione della tabella  
// post: ritorna (hashFun(k, m) + i) mod m, dove hashFun(k, m) = k mod m  
int linearProbing(int k, int i, int m)
```

si implementi la funzione `hashInsert()`:

```
// pre: 0 <= k chiave, T tabella hash correttamente allocata  
// post: ritorna  
//        j se k è inserito in T->array[j]  
//        -1 se k era presente in T  
//        -2 in caso di overflow  
int hashInsert(HashTable T, int k)
```

che inserisce la chiave k nella tabella T .

Nota. Nella visualizzazione dello stato della tabella, il numero -1 è utilizzato per indicare una casella rimasta vuota. Questa visualizzazione, come i messaggi che compaiono nell'output dell'esempio sono generati dalla funzione `test()` e **NON devono essere implementati** nella risposta.

Per esempio:

Test	Risultati
<pre>// test: 1 HashTable T1 = newHashTable(10); int A[] = {7, 15, 34, 12, 35, 27, 5}; int dimA = sizeof(A)/sizeof(int); test(T1, A, dimA);</pre>	chiave 7 inserita in posizione 7 chiave 15 inserita in posizione 5 chiave 34 inserita in posizione 4 chiave 12 inserita in posizione 2 chiave 35 inserita in posizione 6 chiave 27 inserita in posizione 8 chiave 5 inserita in posizione 9 Stato della tabella: -1 -1 12 -1 34 15 35 7 27 5

Risposta (Criteri di penalizzazione: 10, 20, ... %)

Ripristina risposta

L'editor Ace non è pronto. Prova a ricaricare la pagina?
Falling back to raw text area.

```
// pre: 0 <= k chiave, T tabella hash correttamente allocata
// post: ritorna
//      j se k è inserito in T->array[j]
//      -1 se k era presente in T
//      -2 in caso di overflow
int hashInsert(HashTable T, int k) {
    // parte da completare

    T = newHashTable(k);
}
```

Errore/i di sintassi

```
__tester__.c: In function 'hashInsert':
__tester__.c:53:1: error: control reaches end of non-void function [-Werror=return-type]
  53 | }
     | ^
cc1: all warnings being treated as errors
```

Risposta errata

Punteggio di questo invio: 0,00/4,00.

Domanda **3**

Risposta corretta

Punteggio ottenuto 5,00 su 5,00

Decisione se un albero binario è ordinato

Data l'implementazione degli alberi binari con chiavi intere:

```
struct BtreeNd {
    int      key;
    struct BtreeNd* left;
    struct BtreeNd* right;
};

typedef struct BtreeNd* btree;
```

si implementi la funzione:

```
// post: decide se bt e' ordinato (di ricerca) ovvero
//       ritorna true se bt è di ricerca, false altrimenti
bool isOrdered(btree bt)
```

che ritorna true se l'albero bt è ordinato (di ricerca), false altrimenti.

Suggerimento: utilizzando il tipo preimplementato:

```
struct triple {
    bool    isOrdered;
    int     min, max;
};

typedef struct triple Triple;
```

si scriva una funzione ausiliaria che soddisfi le specifiche:

```
// pre: bt != NULL
// post: restituisce la tripla (min, isOrdered, max)
//       dove isOrdered == true se bt e' ordinato, false altrimenti
//       se isOrdered == true allora min e max sono il minimo ed il massimo
//       delle chiavi in bt
Triple isOrderedBTreeAux(btree bt)
```

ATTENZIONE: il Precheck sull'esempio non comporta penalità.

Per esempio:

Test	Risultati
<pre>// test 1 btree bt1 = insert(45, NULL); bt1 = insert(23, bt1); bt1 = insert(52, bt1); bt1 = insert(12, bt1); bt1 = insert(48, bt1); bt1 = insert(1, bt1); bt1 = insert(7, bt1); printf("Albero dato:\n"); printtree(bt1, 0); printTest(isOrdered(bt1));</pre>	<pre>Albero dato: 45 23 12 1 7 52 48 è albero ordinato</pre>

Risposta (Criteri di penalizzazione: 10, 20, ... %)

Ripristina risposta

L'editor Ace non è pronto. Prova a ricaricare la pagina?
Falling back to raw text area.

```
// pre: bt != NULL
// post: restituisce la tripla (min, isOrdered, max)
//       dove isOrdered == true se bt e' ordinato, false altrimenti
//       se isOrdered == true allora min e max sono il minimo ed il massimo
//       delle chiavi in bt
Triple isOrderedBTreeAux(btree bt) {
    // parte da completare
    Triple t;

    if(bt->left == NULL && bt->right == NULL){

        t.isOrdered = true;

        t.min = t.max = bt->key;
    }
    else if(bt->right == NULL){

        Triple tleft = isOrderedBTreeAux(bt->left);

        t.isOrdered = tleft.isOrdered && bt->key > tleft.max;

        t.max = bt->key;
        t.min = tleft.min;
    }
    else if(bt->left == NULL){

        Triple tright = isOrderedBTreeAux(bt->right);

        t.isOrdered = tright.isOrdered && bt->key < tright.min;
```

Tutti i test sono stati superati ✓

Risposta corretta

Punteggio di questo invio: 5,00/5,00.